

University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page Milestone #: 2

Date: February 25, 2026

Group Number: 47

Name	Student Number	CS Alias (Userid)	Preferred E-mail address
Hannah Kamarul Hussain	59502492	d9l7h	hannahjemal123@gmail.com
Mishaa Murugathass	59864777	e6a3s	mur_u_mish@hotmail.com
Rojin Khashei	81018566	n8u5z	rojin.kv@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

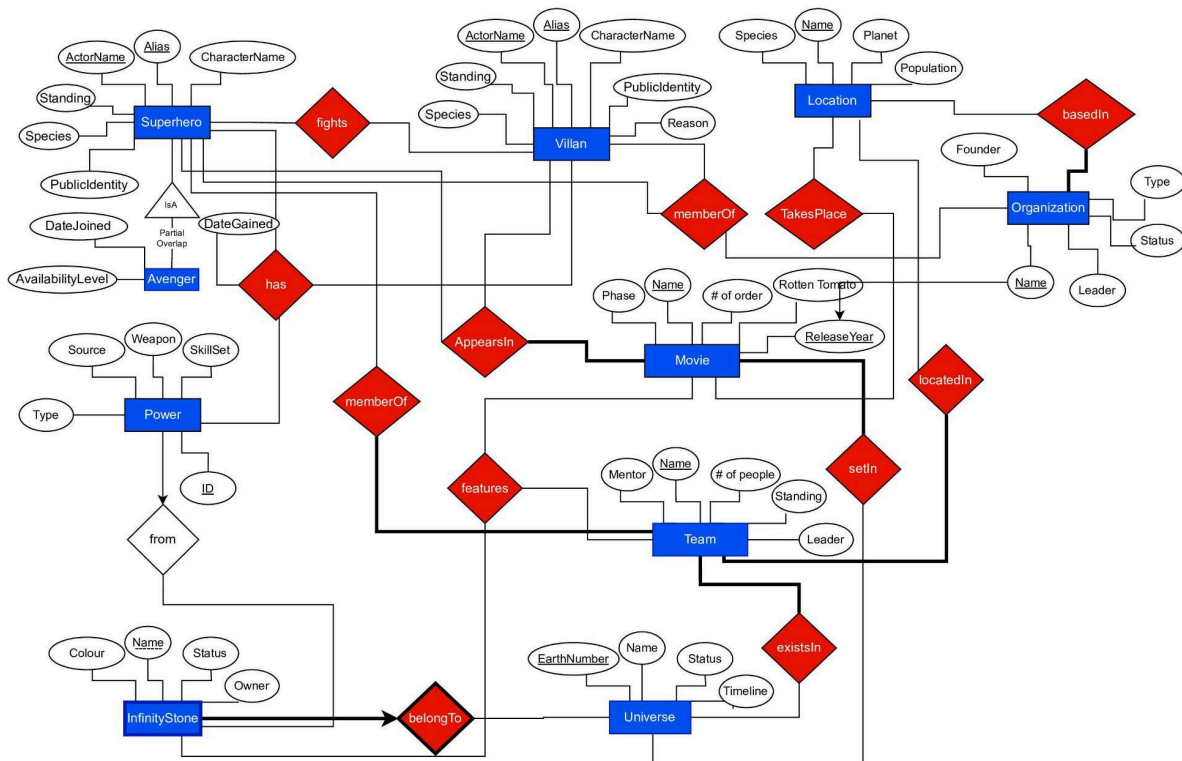
Short Project Description (~2-3 sentences):

The domain of this application is the Marvel cinematic universe and related elements from the comic universe. It focuses on organizing information about superheroes and villains, including their powers, team or organizational information, and the universes they exist in (such as Earth-616). The database also captures how these elements relate to one another, such as the connections between characters and their powers or their membership in groups like the Avengers or S.H.I.E.L.D.

Technology Choice:

This project will use Oracle and JavaScript.

Corrected ER Diagram (same as milestone 1):



Note: Milestone 1 was graded 18/18 with no feedback, so no changes have been made to the original ER diagram.

Relational Schema (Derived from ER diagram):

Entity Tables: Underlined are Primary keys and bolded are **foreign keys**. All primary keys have the constraint to be not null and unique. Unless listed there are not any additional candidate keys or other constraints.

Superhero(ActorName: VARCHAR(50), Alias: VARCHAR(50), CharacterName: VARCHAR(50), Standing: VARCHAR(30), Species: VARCHAR(30), PublicIdentity: VARCHAR(30))

Avenger(**ActorName**: VARCHAR(50), **Alias**: VARCHAR(50), AvailabilityLevel: INTEGER, DateJoined: INTEGER)

- FK: (ActorName, Alias) REFERENCES Superhero(ActorName, Alias)

Universe (EarthNumber: INT, Name: VARCHAR(20), Status: VARCHAR(20), Timeline: VARCHAR(20))

InfinityStone(**EarthNumber**: INTEGER, Name: VARCHAR(50), Status: VARCHAR(50), Owner: VARCHAR(50), Colour: VARCHAR(50))

- FK: EarthNumber REFERENCES Universe(EarthNumber)

Villain(ActorName: VARCHAR(50), Alias: VARCHAR(50), CharacterName: VARCHAR(50), Standing: VARCHAR(30), Species: VARCHAR(30), PublicIdentity: VARCHAR(30), Reason: VARCHAR(30))

Power(ID: INT, Type: VARCHAR(30), SkillSet: VARCHAR(30), Weapon: VARCHAR(30), Source: VARCHAR(30))

Team(Name: VARCHAR(50), NumberOfPeople: INT, Standing: VARCHAR(50), Mentor: VARCHAR(50), Leader: VARCHAR(50))

Organization(Name: VARCHAR(50), Leader: VARCHAR(50), Status: VARCHAR(50), Type: VARCHAR(50), Founder: VARCHAR(50))

Movie(Name: VARCHAR(50), ReleaseYear: INTEGER, NumberOfOrder: INTEGER, Phase: INTEGER, RottenTomato: INTEGER)

Location(Name: VARCHAR(50), Species: VARCHAR(50), Planet: VARCHAR(50), Population: VARCHAR(50))

Relationship Tables: Underlined are Primary keys and bolded are **foreign keys**. All primary keys have the constraint to be not null and unique. Unless listed there are not any additional candidate keys or other constraints.

Fights(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50))

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)

SuperheroHasPower(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **PowerID**: INTEGER, DateGained: DATE)

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: PowerID REFERENCES Power(ID)

VillainHasPower(**VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50), **PowerID**: INTEGER, DateGained: DATE)

- FK1: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
- FK2: PowerID REFERENCES Power(ID)

MemberOfTeam(**ActorName**: VARCHAR(50), **Alias**: VARCHAR(50), **Name**: VARCHAR(50))

- FK1: (ActorName, Alias) REFERENCES Superhero(ActorName, Alias)
- FK2: Name REFERENCES Team(Name)
- Constraint: **Note**: Not representable as many-many relationship, and team side has a total participation constraint

AppearsIn(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50), **MovieName**: VARCHAR(50), **MovieReleaseYear**: INTEGER)

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
- FK3: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)
- Constraints: **Note**: Not representable as many-many relationship, and movie side has a total participation constraint

PowerFrom(**PowerID**: INTEGER, **StoneName**: VARCHAR(50), **EarthNumber**: INTEGER)

- FK1: PowerID REFERENCES Power(ID)
- FK2: StoneName REFERENCES InfinityStone(Name)

Features(**EarthNum**: INTEGER, **InfinityStoneName**: VARCHAR(50), **TeamName**: VARCHAR(50), **MovieName**: VARCHAR(50), **MovieReleaseYear**: INT)

- FK1: (EarthNum InfinityStoneName) REFERENCES InfinityStone(EarthNumber, Name)
- FK2: TeamName REFERENCES Team(Name)
- FK3: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)

ExistsIn(**TeamName**: VARCHAR(50), **EarthNumber**: INTEGER)

- FK1: TeamName REFERENCES Team(Name)
- FK2: EarthNumber REFERENCES Universe(EarthNumber)

- Constraints: **Note**: Not representable as many-many relationship, and team side has a total participation constraint

SetIn(**EarthNumber**: INTEGER, **MovieName**: VARCHAR(50), **MovieReleaseYear**: INTEGER)

- FK1: EarthNumber REFERENCES Universe(EarthNumber)
- FK2: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)
- Constraints: **Note**: Not representable as many-many relationship, and movie side has a total participation constraint

LocatedIn(**TeamName**: VARCHAR(50), **LocationName**: VARCHAR(50))

- FK1: TeamName REFERENCES Team(Name)
- FK2: LocationName REFERENCES Location(Name)
- Constraints: **Note**: Not representable as many-many relationship, and team side has a total participation constraint

MemberOfOrganization(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50), **OrganizationName**: VARCHAR(50))

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
- FK3: OrganizationName REFERENCES Organization(Name)

TakesPlace(**MovieName**: VARCHAR(50), **MovieReleaseYear**: INTEGER, **LocationName**: VARCHAR(50))

- FK1: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)
- FK2: LocationName REFERENCES Location(Name)

BasedIn(**LocationName**: VARCHAR(50), **OrganizationName**: VARCHAR(50))

- FK1: LocationName REFERENCES Location(Name)
- FK2: OrganizationName REFERENCES Organization(Name)
- Constraints: **Note**: Not representable as many-many relationship, and organization side has a total participation constraint

Functional Dependencies:

Superhero:

- PK FD: (ActorName, Alias) → CharacterName, Standing, Species, PublicIdentity (**BCNF**)

Avenger:

- PK FD: (ActorName, Alias) → AvailabilityLevel, DateJoined (**BCNF**)

Villain:

- PK FD: (ActorName, Alias) → CharacterName, Standing, Species, PublicIdentity, Reason (**BCNF**)

Universe:

- PK FD: EarthNumber → Name, Status, Timeline (**BCNF**)
- CK FD: Name → EarthNumber, Status, Timeline

InfinityStone

- PK FD: (Name, EarthNumber) → Status, Owner, Colour
- Non-PK /CK FD: Name → Colour

Power

- PK FD: ID → Type, SkillSet, Weapon, Source
- Non-PK/CK FD: Weapon → Source

Team

- PK FD: Name → NumberOfPeople, Standing, Mentor, Leader (**BCNF**)

Organization

- PK FD: Name → Leader, Status, Type, Founder (**BCNF**)

Movie

- PK FD: (Name, ReleaseYear) → NumberOfOrder, Phase, RottenTomato
- Non PK/CK FD: NumberOfOrder → Phase

Location

- PK FD: Name → Species, Planet, Population (**BCNF**)

SuperheroHas

- PK FD: (HeroActorName, HeroAlias, ID) → DateGained (**BCNF**)

VillainHas

- PK FD: (VillainActorName, VillainAlias, ID) → DateGained (**BCNF**)

Relationship relations with only key attributes:

Fights, MemberOfTeam, AppearsIn, Features, ExistsIn, SetIn, LocatedIn, MemberOfOrganization, TakesPlace, BasedIn. These relations contain only primary key attributes and no non-key attributes. Therefore, they do not have non-trivial functional dependencies involving non-key attributes. The only dependencies are trivial/key dependencies implied by the primary key.

Normalization (3NF or BCNF): The ones that have to be normalized due to non PK/CK FDs:

Normalization of InfinityStone:

Original Relation:

- InfinityStone(EarthNumber: INTEGER, Name: VARCHAR(50), Status: VARCHAR(50), Owner: VARCHAR(50), Colour: VARCHAR(50))
 - FK: EarthNumber REFERENCES Universe(EarthNumber)

FD's:

- PK FD: (Name, EarthNumber) → Status, Owner, Colour
- Non-PK /CK FD: Name → Colour

Reason for Decomposition:

- Name → Colour violates BCNF because Name is not a superkey of InfinityStone (the PK is (Name, EarthNumber)).

Decomposition:

1. StoneColour(Name: VARCHAR(50), Colour: VARCHAR(50))
2. InfinityStone(EarthNumber: INTEGER, Name: VARCHAR(50), Status: VARCHAR(50), Owner: VARCHAR(50))

Final Normalized Tables:

- StoneColour(Name: VARCHAR(50), Colour: VARCHAR(50))
- InfinityStone(EarthNumber: INTEGER, Name: VARCHAR(50), Status: VARCHAR(50), Owner: VARCHAR(50))
 - FK1: EarthNumber REFERENCES Universe(EarthNumber)
 - FK2: Name REFERENCES StoneColour(Name)

Normalization of Power:

Original relation:

- Power(ID: INTEGER, Type: VARCHAR(30), SkillSet: VARCHAR(30), Weapon: VARCHAR(30), Source: VARCHAR(30))

FDs:

- PK FD: ID $\rightarrow$ Type, SkillSet, Weapon, Source
- Non-PK/CK FD: Weapon $\rightarrow$ Source

Reason for Decomposition:

- (Name, EarthNumber) $\rightarrow$ Source, Type violates BCNF because Weapon is not a superkey (the PK is ID).

Decomposition:

1. WeaponSource(Weapon: VARCHAR(30), Source: VARCHAR(30))
2. Power(ID: INTEGER, Type: VARCHAR(30), SkillSet: VARCHAR(30), Weapon: VARCHAR(30))

Final Normalized Tables

- WeaponSource(Weapon: VARCHAR(30), Source: VARCHAR(30))
- Power(ID: INTEGER, Type: VARCHAR(30), SkillSet: VARCHAR(30), **Weapon**: VARCHAR(30))
 - FK: Weapon REFERENCES WeaponSource(Weapon)

Normalization of Movie:

Original relation:

- Movie(Name: VARCHAR(50), ReleaseYear: INTEGER, NumberOfOrder: INTEGER, Phase: INTEGER, RottenTomato: INTEGER)

FDs:

- PK FD: (Name, ReleaseYear) $\rightarrow$ NumberOfOrder, Phase, RottenTomato
- Non PK/CK FD: NumberOfOrder $\rightarrow$ Phase

Reason for Decomposition:

- NumberOfOrder $\rightarrow$ Phase violates BCNF because NumberOfOrder is not a superkey of Movie (the PK is (Name, ReleaseYear)).

Decomposition:

- Decompose Movie using NumberOfOrder $\rightarrow$ Phase into:
 1. MovieOrderPhase(NumberOfOrder: INTEGER, Phase: INTEGER)
 2. Movie(Name: VARCHAR(50), ReleaseYear: INTEGER, NumberOfOrder: INTEGER, RottenTomato: INTEGER)

Final Normalized Tables

- MovieOrderPhase(NumberOfOrder: INTEGER, Phase: INTEGER)
- Movie(Name: VARCHAR(50), ReleaseYear: INTEGER, **NumberOfOrder**: INTEGER, RottenTomato: INTEGER)

- FK: NumberOfOrder REFERENCES MovieOrderPhase(NumberOfOrder)

Finalized Tables After Normalization: If no candidate keys available, it will not be listed.

Changed relations:

StoneColour(Name: VARCHAR(50), Colour: VARCHAR(50))

InfinityStone(EarthNumber: INTEGER, Name: VARCHAR(50), Status: VARCHAR(50), Owner: VARCHAR(50))

- FK1: EarthNumber REFERENCES Universe(EarthNumber)
- FK2: Name REFERENCES StoneColour(Name)

WeaponSource(Weapon: VARCHAR(30), Source: VARCHAR(30))

Power(ID: INTEGER, Type: VARCHAR(30), SkillSet: VARCHAR(30), **Weapon**: VARCHAR(30))

- FK: Weapon REFERENCES WeaponSource(Weapon)

MovieOrderPhase(NumberOfOrder: INTEGER, Phase: INTEGER)

Movie(Name: VARCHAR(50), ReleaseYear: INTEGER, **NumberOfOrder**: INTEGER, RottenTomato: INTEGER)

- FK: NumberOfOrder REFERENCES MovieOrderPhase(NumberOfOrder)

The following relations are already in 3NF/BCNF and remain unchanged:

Superhero(ActorName: VARCHAR(50), Alias: VARCHAR(50), CharacterName: VARCHAR(50), Standing: VARCHAR(30), Species: VARCHAR(30), PublicIdentity: VARCHAR(30))

Avenger(ActorName: VARCHAR(50), Alias: VARCHAR(50), AvailabilityLevel: INTEGER, DateJoined: INTEGER)

- FK: (ActorName, Alias) REFERENCES Superhero(ActorName, Alias)

Universe (EarthNumber: INTEGER, Name: VARCHAR(20), Status: VARCHAR(20), Timeline: VARCHAR(20))

Villain(ActorName: VARCHAR(50), Alias: VARCHAR(50), CharacterName: VARCHAR(50), Standing: VARCHAR(30), Species: VARCHAR(30), PublicIdentity: VARCHAR(30), Reason: VARCHAR(30))

Team(Name: VARCHAR(50), NumberOfPeople: INTEGER, Standing: VARCHAR(50), Mentor: VARCHAR(50), Leader: VARCHAR(50))

Organization(Name: VARCHAR(50), Leader: VARCHAR(50), Status: VARCHAR(50), Type: VARCHAR(50), Founder: VARCHAR(50))

Location(Name: VARCHAR(50), Species: VARCHAR(50), Planet: VARCHAR(50), Population: VARCHAR(50))

Relationship Tables:

Fights(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50))

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
-

SuperheroHasPower(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **PowerID**: INTEGER, DateGained: DATE)

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: PowerID REFERENCES Power(ID)

VillainHasPower(**VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50), **PowerID**: INTEGER, DateGained: DATE)

- FK1: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
- FK2: PowerID REFERENCES Power(ID)

MemberOfTeam(**ActorName**: VARCHAR(50), **Alias**: VARCHAR(50), **Name**: VARCHAR(50))

- FK1: (ActorName, Alias) REFERENCES Superhero(ActorName, Alias)
- FK2: Name REFERENCES Team(Name)
- Constraint: **Note**: Not representable as many-many relationship, and team side has a total participation constraint

AppearsIn(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50), **MovieName**: VARCHAR(50), **MovieReleaseYear**: INTEGER)

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
- FK3: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)
- Constraints: **Note**: Not representable as many-many relationship, and movie side has a total participation constraint

PowerFrom(**PowerID**: INTEGER, **StoneName**: VARCHAR(50))

- FK1: PowerID REFERENCES Power(ID)
- FK2: StoneName REFERENCES StoneColour(Name)
- Constraints: StoneName NOT NULL

Features(**EarthNum**: INTEGER, **InfinityStoneName**: VARCHAR(50), **TeamName**: VARCHAR(50), **MovieName**: VARCHAR(50), **MovieReleaseYear**: INT)

- FK1: (EarthNum InfinityStoneName) REFERENCES InfinityStone(EarthNumber, Name)
- FK2: TeamName REFERENCES Team(Name)
- FK3: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)

ExistsIn(**TeamName**: VARCHAR(50), **EarthNumber**: INTEGER)

- FK1: TeamName REFERENCES Team(Name)

- FK2: EarthNumber REFERENCES Universe(EarthNumber)
- Constraints: **Note**: Not representable as many-many relationship, and team side has a total participation constraint

SetIn(**EarthNumber**: INTEGER, **MovieName**: VARCHAR(50), **MovieReleaseYear**: INTEGER)

- FK1: EarthNumber REFERENCES Universe(EarthNumber)
- FK2: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)
- Constraints: **Note**: Not representable as many-many relationship, and movie side has a total participation constraint

LocatedIn(**TeamName**: VARCHAR(50), **LocationName**: VARCHAR(50))

- FK1: TeamName REFERENCES Team(Name)
- FK2: LocationName REFERENCES Location(Name)
- Constraints: **Note**: Not representable as many-many relationship, and team side has a total participation constraint

MemberOfOrganization(**HeroActorName**: VARCHAR(50), **HeroAlias**: VARCHAR(50), **VillainActorName**: VARCHAR(50), **VillainAlias**: VARCHAR(50), **OrganizationName**: VARCHAR(50))

- FK1: (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias)
- FK2: (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias)
- FK3: OrganizationName REFERENCES Organization(Name)

TakesPlace(**MovieName**: VARCHAR(50), **MovieReleaseYear**: INTEGER, **LocationName**: VARCHAR(50))

- FK1: (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear)
- FK2: LocationName REFERENCES Location(Name)

BasedIn(**LocationName**: VARCHAR(50), **OrganizationName**: VARCHAR(50))

- FK1: LocationName REFERENCES Location(Name)
- FK2: OrganizationName REFERENCES Organization(Name)
- Constraints: **Note**: Not representable as many-many relationship, and organization side has a total participation constraint

SQL DDL Statements:

```
CREATE TABLE Superhero (
    ActorName      VARCHAR(50),
    Alias          VARCHAR(50),
    CharacterName  VARCHAR(50),
    Standing       VARCHAR(30),
    Species        VARCHAR(30),
    PublicIdentity VARCHAR(30),
    PRIMARY KEY    (ActorName, Alias));
```

```
CREATE TABLE Villain (  
    ActorName          VARCHAR(50),  
    Alias              VARCHAR(50),  
    CharacterName      VARCHAR(50),  
    Standing           VARCHAR(30),  
    Species            VARCHAR(30),  
    PublicIdentity     VARCHAR(30),  
    Reason             VARCHAR(30),  
    PRIMARY KEY (ActorName, Alias));
```

```
CREATE TABLE Universe (  
    EarthNumber        INTEGER,  
    Name               VARCHAR(20),  
    Status             VARCHAR(20),  
    Timeline           VARCHAR(20),  
    PRIMARY KEY (EarthNumber));
```

```
CREATE TABLE Team (  
    Name               VARCHAR(50),  
    NumberOfPeople    INTEGER,  
    Standing           VARCHAR(50),  
    Mentor            VARCHAR(50),  
    Leader            VARCHAR(50),  
    PRIMARY KEY (Name));
```

```
CREATE TABLE Organization (  
    Name               VARCHAR(50),  
    Leader            VARCHAR(50),  
    Status            VARCHAR(50),  
    Type              VARCHAR(50),  
    Founder           VARCHAR(50),  
    PRIMARY KEY (Name));
```

```
CREATE TABLE Location (  
    Name               VARCHAR(50),  
    Species            VARCHAR(50),  
    Planet            VARCHAR(50),  
    Population         VARCHAR(50),  
    PRIMARY KEY (Name));
```

```
CREATE TABLE StoneColour (  

```

Name VARCHAR(50),
Colour VARCHAR(50),
PRIMARY KEY (Name));

CREATE TABLE MovieOrderPhase (
NumberOfOrder INTEGER,
Phase INTEGER,
PRIMARY KEY (NumberOfOrder));

CREATE TABLE InfinityStone (
EarthNumber INTEGER,
Name VARCHAR(50),
Status VARCHAR(50),
Owner VARCHAR(50),
PRIMARY KEY (Name, EarthNumber),
FOREIGN KEY (EarthNumber) **REFERENCES** Universe(EarthNumber),
FOREIGN KEY (Name) **REFERENCES** StoneColour(Name));

CREATE TABLE Movie (
Name VARCHAR(50),
ReleaseYear INTEGER,
NumberOfOrder INTEGER,
RottenTomato INTEGER,
PRIMARY KEY (Name, ReleaseYear),
FOREIGN KEY (NumberOfOrder) **REFERENCES** MovieOrderPhase(NumberOfOrder));

CREATE TABLE Avenger (
ActorName VARCHAR(50),
Alias VARCHAR(50),
AvailabilityLevel INTEGER,
DateJoined INTEGER,
PRIMARY KEY (ActorName, Alias),
FOREIGN KEY (ActorName, Alias) **REFERENCES** Superhero(ActorName, Alias));

CREATE TABLE WeaponSource (
Weapon VARCHAR(30),
Source VARCHAR(30),
PRIMARY KEY (Weapon));

CREATE TABLE Power (
ID INTEGER,
SkillSet VARCHAR(30),
Weapon VARCHAR(30),

Type VARCHAR(30),
PRIMARY KEY (ID),
FOREIGN KEY (Weapon) **REFERENCES** WeaponSource(Weapon));

CREATE TABLE Fights (
 HeroActorName VARCHAR(50),
 HeroAlias VARCHAR(50),
 VillainActorName VARCHAR(50),
 VillainAlias VARCHAR(50),
 PRIMARY KEY (HeroActorName, HeroAlias, VillainActorName, VillainAlias),
 FOREIGN KEY (HeroActorName, HeroAlias) **REFERENCES** Superhero(ActorName, Alias),
 FOREIGN KEY (VillainActorName, VillainAlias) **REFERENCES** Villain(ActorName, Alias));

CREATE TABLE VillainHasPower (
 PowerID INTEGER,
 VillainActorName VARCHAR(50),
 VillainAlias VARCHAR(50),
 DateGained DATE,
 PRIMARY KEY (VillainActorName, VillainAlias, PowerID),
 FOREIGN KEY (VillainActorName, VillainAlias) **REFERENCES** Villain(ActorName, Alias),
 FOREIGN KEY (PowerID) **REFERENCES** Power(ID));

CREATE TABLE HeroHasPower (
 HeroActorName VARCHAR(50),
 HeroAlias VARCHAR(50),
 PowerID INTEGER,
 DateGained DATE,
 PRIMARY KEY (HeroActorName, HeroAlias, PowerID),
 FOREIGN KEY (HeroActorName, HeroAlias) **REFERENCES** Superhero(ActorName, Alias),
 FOREIGN KEY (PowerID) **REFERENCES** Power(ID));

CREATE TABLE MemberOfTeam (
 ActorName VARCHAR(50),
 Alias VARCHAR(50),
 Name VARCHAR(50),
 PRIMARY KEY (ActorName, Alias, Name),
 FOREIGN KEY (ActorName, Alias) **REFERENCES** Superhero(ActorName, Alias),
 FOREIGN KEY (Name) **REFERENCES** Team(Name));

CREATE TABLE AppearsIn (
 HeroActorName VARCHAR(50),
 HeroAlias VARCHAR(50),
 VillainActorName VARCHAR(50),

```

VillainAlias          VARCHAR(50),
MovieName             VARCHAR(50),
MovieReleaseYear      INTEGER,
PRIMARY KEY (
    HeroActorName, HeroAlias, VillainActorName, VillainAlias, MovieName, MovieReleaseYear),
FOREIGN KEY (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias),
FOREIGN KEY (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias),
FOREIGN KEY (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear));

```

```

CREATE TABLE Features (
    EarthNum           INTEGER,
    InfinityStoneName  VARCHAR(50),
    TeamName           VARCHAR(50),
    MovieName          VARCHAR(50),
    MovieReleaseYear   INTEGER,
PRIMARY KEY (EarthNum, InfinityStoneName, TeamName, MovieName, MovieReleaseYear),
FOREIGN KEY (InfinityStoneName, EarthNum)
    REFERENCES InfinityStone(Name, EarthNumber),
FOREIGN KEY (TeamName) REFERENCES Team(Name),
FOREIGN KEY (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear));

```

```

CREATE TABLE ExistsIn (
    TeamName           VARCHAR(50),
    EarthNumber        INTEGER,
PRIMARY KEY (TeamName, EarthNumber),
FOREIGN KEY (TeamName) REFERENCES Team(Name),
FOREIGN KEY (EarthNumber) REFERENCES Universe(EarthNumber));

```

```

CREATE TABLE SetIn (
    EarthNumber        INTEGER,
    MovieName          VARCHAR(50),
    MovieReleaseYear   INTEGER,
PRIMARY KEY (EarthNumber, MovieName, MovieReleaseYear),
FOREIGN KEY (EarthNumber) REFERENCES Universe(EarthNumber),
FOREIGN KEY (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear));

```

```

CREATE TABLE LocatedIn (
    TeamName           VARCHAR(50),
    LocationName       VARCHAR(50),
PRIMARY KEY (TeamName, LocationName),
FOREIGN KEY (TeamName) REFERENCES Team(Name),
FOREIGN KEY (LocationName) REFERENCES Location(Name));

```

```

CREATE TABLE MemberOfOrganization (
    HeroActorName      VARCHAR(50),
    HeroAlias           VARCHAR(50),
    VillainActorName    VARCHAR(50),
    VillainAlias        VARCHAR(50),
    OrganizationName    VARCHAR(50),
    PRIMARY KEY (HeroActorName, HeroAlias, VillainActorName, VillainAlias, OrganizationName),
    FOREIGN KEY (HeroActorName, HeroAlias) REFERENCES Superhero(ActorName, Alias),
    FOREIGN KEY (VillainActorName, VillainAlias) REFERENCES Villain(ActorName, Alias),
    FOREIGN KEY (OrganizationName) REFERENCES Organization(Name));

```

```

CREATE TABLE TakesPlace (
    MovieName          VARCHAR(50),
    MovieReleaseYear    INTEGER,
    LocationName        VARCHAR(50),
    PRIMARY KEY (MovieName, MovieReleaseYear, LocationName),
    FOREIGN KEY (MovieName, MovieReleaseYear) REFERENCES Movie(Name, ReleaseYear),
    FOREIGN KEY (LocationName) REFERENCES Location(Name));

```

```

CREATE TABLE BasedIn (
    LocationName        VARCHAR(50),
    OrganizationName    VARCHAR(50),
    PRIMARY KEY (LocationName, OrganizationName),
    FOREIGN KEY (LocationName) REFERENCES Location(Name),
    FOREIGN KEY (OrganizationName) REFERENCES Organization(Name));

```

```

CREATE TABLE PowerFrom (
    PowerID INTEGER,
    StoneName VARCHAR(50),
    EarthNumber INTEGER,
    PRIMARY KEY (PowerID, StoneName, EarthNumber),
    FOREIGN KEY (PowerID) REFERENCES Power(ID),
    FOREIGN KEY (StoneName, EarthNumber) REFERENCES InfinityStone(Name, EarthNumber));

```

AI Statement: None of this project was completed with the help of AI.